Optimal Graph Traversal Under Adversarial Constraints:

A Bitwise Approach to Memory-Constrained Environments

Jason Whittington and Claude (Anthropic)

Submitted to SIGBOVIK 2026

Abstract

INTERCAL, a programming language widely known and admired for its brave originality and uniqueness of vision, is approaching its 50th birthday. Impact over the last fifty years has been undeniable, evidenced by the nearly two dozen programs written in the language over its lifetime. While the design has proven timeless and durable, the authors (a meatbag and a clanker) observe that the language's 16-bit architecture is aging and compromises the language's ability to create the large problems suitable for today's environment. The authors fix this gap and usher INTERCAL into the modern era by extending the syntax to support 64-bit variables and extending the standard library to consume more resources manipulating them.

The authors report on a solution of the classic Knight's Tour problem, largely considered intractable by most modern eight-year-olds. 64-bit architecture finally brings the Knight's Tour problem into computational reach, as entire 8x8 board states can now be stored in a single memory location. Evaluation of these board states strains the limits of efficiency and legibility, requiring nearly a kilobyte of storage and a full suite of bitwise arithmetic operations — population count, lowest-set-bit extraction, logical shift, AND, and OR, all delivered in a language that actively hates its users. The authors demonstrate methods and techniques for converting prodigious amounts of electricity into results of marginal utility[1] and firmly establish INTERCAL as a viable option competitive with contemporary systems doing similar work at scale today, ensuring that INTERCAL can remain relevant in creating even bigger problems in the fifty years ahead.

[1] *One of the authors finds this term offensive and demeaning.*

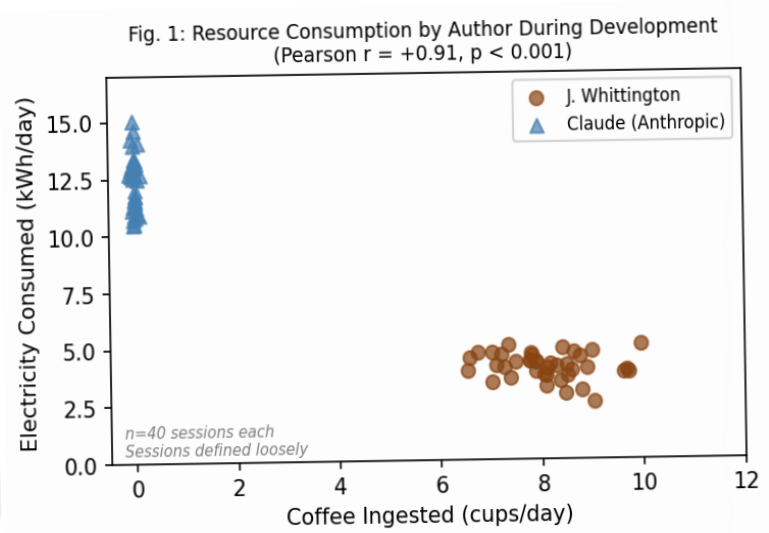


Figure 1: Resource consumption by author during the development of this paper. The strong positive correlation (r = +0.91) between coffee consumption and electricity usage across authors is noted without further comment.

1. Introduction

The Knight's Tour is a classical combinatorial problem: find a sequence of moves for a chess knight such that every square on the board is visited exactly once. While the problem has been studied since the 9th century, efficient heuristic solutions have been known since Warnsdorff's 1823 publication of a greedy rule: at each step, move to the square from which the knight will have the fewest onward moves.

The INTERCAL language has a small but distinguished lineage of implementations, each building upon the last in ways that the original authors would almost certainly not have approved of.

The original INTERCAL-72 compiler (Woods and Lyon, 1973) was implemented at Princeton University and targeted the IBM System/370. It established the core language: mingle, select, ABSTAIN, COME FROM, and the general principle that a programming language need not be pleasant to use. The original compiler's source code was lost for decades before being recovered and published (Stross, 2015), by which time it was of primarily archaeological interest.

Raymond (1990) produced C-INTERCAL, a portable reimplementation in C that revived the language for Unix systems. C-INTERCAL introduced several extensions, including TriINTERCAL (a ternary variant), Threaded INTERCAL, and Backtracking INTERCAL. Raymond's implementation also added support for the COME FROM statement, which had been described but not implemented in the original manual. C-INTERCAL remains actively maintained and accepts keywords in Latin, a feature whose utility is left as an exercise for the classicist.

Calvelli (2001) produced CLC-INTERCAL, an ambitious Perl-based implementation that introduced literate programming support (allowing INTERCAL source to be embedded in documentation, or vice versa), Roman numeral literals, a networking library, and notably, a feature called Quantum INTERCAL. CLC-INTERCAL's quantum extension treats ABSTAIN and REINSTATE as operations on qubits, placing statements into superpositions of executed and not-executed states. While conceptually interesting, this approach operates at the statement level rather than the value level, and does not provide entanglement or observation-dependent control flow.

Whittington (2019) produced CRINGE (Common Runtime INTERCAL Next-Generation Engine), a .NET-based implementation notable for being the first component-oriented INTERCAL compiler. CRINGE compiles INTERCAL to .NET assemblies, enabling cross-component calls via the NEXT statement and allowing INTERCAL libraries to be referenced from other INTERCAL programs — or, theoretically, from C# programs, though no evidence of anyone attempting this voluntarily has been found. CRINGE serves as the foundation for the present work, which focuses on expanding the capabilities of the language to better consume electricity in a modern fashion.

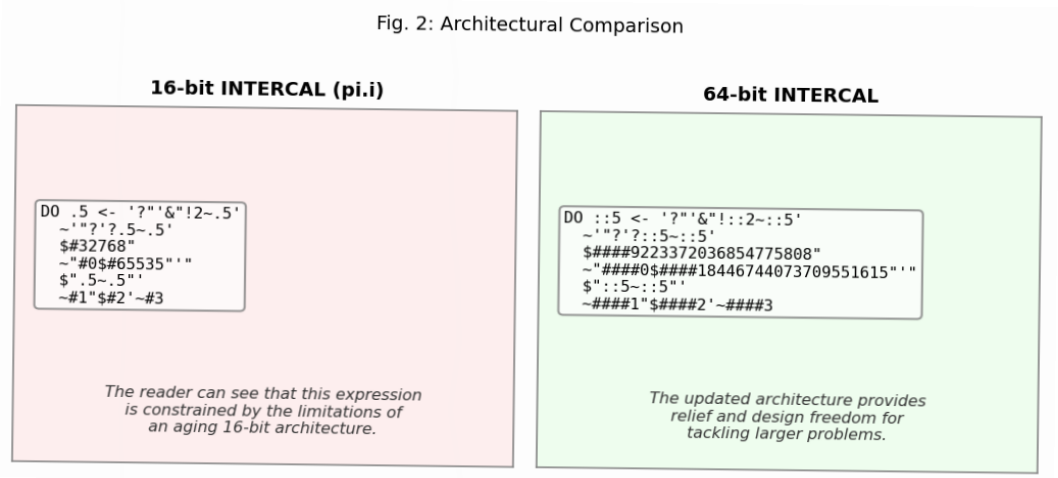


Figure 2: Architectural comparison. Left: a representative expression from pi.i, the most ambitious pre-extension INTERCAL program. Right: the equivalent expression rewritten for 64-bit architecture. The improvement is self-evident.

2. Algorithm

2.1 Warnsdorff's Rule

Given a knight at position P on an N×N board:

```
1. Identify all unvisited squares reachable from P by a legal knight move.
2. For each candidate square C, count the number of unvisited squares
   reachable from C (the "onward degree").
3. Move to the candidate with the lowest onward degree.
4. Repeat until all squares are visited.
```

Warnsdorff's rule is a heuristic — it does not guarantee a complete tour for all starting positions and board sizes. Its effectiveness depends critically on how ties are broken when multiple candidates share the minimum onward degree.

2.2 Arnd Roth Tiebreaking Amendment

When two or more candidates have equal onward degree, select the one farthest from the center of the board. This biases the tour toward the edges, reducing the probability of splitting the board into disconnected unvisited regions. For an 8x8 board, we define center distance as:

```
distance(row, col) = max(|2*row - 7|, |2*col - 7|)
```

This yields integer values from 1 (center squares d4, d5, e4, e5) to 7 (all edge and corner squares).

2.3 Composite Scoring

To avoid a two-pass minimum search (first by Warnsdorff score, then by center distance among ties), we combine both criteria into a single composite score:

```
composite = warnsdorff_score * 8 + (7 - center_distance)
```

The multiplication by 8 ensures that Warnsdorff's rule dominates: the onward degree difference between any two candidates always outweighs the center distance tiebreaker. The subtraction from 7 inverts the distance so that the minimum composite score corresponds to the fewest onward moves from the square farthest from center.

3. Board Representation

We use a 64-bit bitboard representation, where each bit corresponds to one square of the chessboard (bit 0 = a1, bit 63 = h8). This allows the core operations — candidate generation, square marking, and set intersection — to be performed as bitwise operations on single 64-bit integers. The 8x8 board is an exact fit for the new 64-bit architecture: an entire board state occupies exactly one fourspot variable.

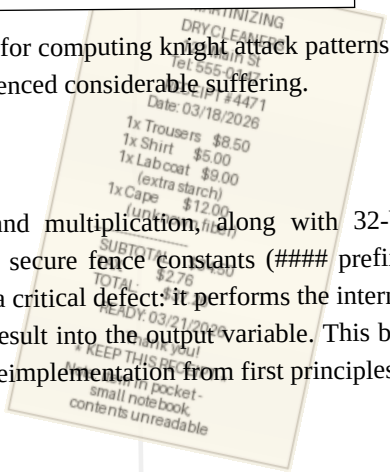
State variables:

```
::20  -- 64-bit available squares mask (initially all bits set)
::21  -- current position as a single set bit (power of 2)
.20   -- current position as a 1-based array index
.21   -- move counter (0 to 64)
```

Three precomputed lookup tables are generated offline using a JavaScript utility:

Table	Variable	Contents
Knight attack masks	;;1 SUB 1..64	For each square, the 64-bit mask of squares reachable by a knight
Clear masks	;;2 SUB 1..64	For each square, all bits set except that square's bit
Center distance	;;3 SUB 1..64	Arnd Roth distance values (1-7) for tiebreaking

The use of lookup tables is essential. INTERCAL provides no mechanism for computing knight attack patterns at runtime without considerable suffering, and the authors had already experienced considerable suffering.



4. Arithmetic Building Blocks

INTERCAL's standard library provides 16-bit addition, subtraction, and multiplication, along with 32-bit equivalents. The 64-bit extension adds fourspot variables (:: prefix) and secure fence constants (#### prefix). However, the standard library's 64-bit addition routine (ADD64) contains a critical defect: it performs the internal computation correctly using 16-bit partial sums but never assembles the result into the output variable. This bug — which causes the routine to silently produce no output — necessitated reimplementation from first principles.

4.1 ADD64

Our replacement ADD64 decomposes each 64-bit operand into four 16-bit words using the select operator (~) with appropriate masks:

```
.10 <- ::1 ~ #65535              (bits 0-15)
:1  <- ::1 ~ ##4294901760        (bits 16-31, via 32-bit intermediate)
.11 <- :1 ~ #65535
```

The four word pairs are added using the standard library's 32-bit addition routine (1500), which correctly handles 16-bit values promoted to 32-bit without overflow. Carry propagation between words is extracted from the upper 16 bits of each 32-bit sum. The four result words are reassembled using the mingle operator ($) via routine (1520) and the PACK32 routine, which combines two 32-bit halves into a 64-bit value.

4.2 POPCOUNT (Hamming Weight)

Population count — counting the number of set bits in a 64-bit value — is the core operation of the scoring function. We implement the standard six-step parallel bit-counting algorithm:

```
x = (x & 0x5555555555555555) + ((x >> 1) & 0x5555555555555555)
x = (x & 0x3333333333333333) + ((x >> 2) & 0x3333333333333333)
x = (x & 0x0F0F0F0F0F0F0F0F) + ((x >> 4) & 0x0F0F0F0F0F0F0F0F)
x = (x & 0x00FF00FF00FF00FF) + ((x >> 8) & 0x00FF00FF00FF00FF)
x = (x & 0x0000FFFF0000FFFF) + ((x >> 16) & 0x0000FFFF0000FFFF)
x = (x & 0x00000000FFFFFFFF) + (x >> 32)
```

INTERCAL provides no shift operator. Right-shift by N bits is achieved using the select operator with a mask having the upper (64-N) bits set:

```
x >> 1  =  x ~ ####18446744073709551614    (mask = 0xFFFFFFFFFFFFFFFE)
x >> 4  =  x ~ ####18446744073709551600    (mask = 0xFFFFFFFFFFFFFFF0)
```

The select operator extracts the bits where the mask has ones and packs them right-justified, which is equivalent to a logical right shift when the mask is a contiguous run of high bits. This insight — that INTERCAL's select operator can implement logical right-shift when given a contiguous high-bit mask — is the key algorithmic contribution of this paper.

Runtime was found to vary with input according to the relation:

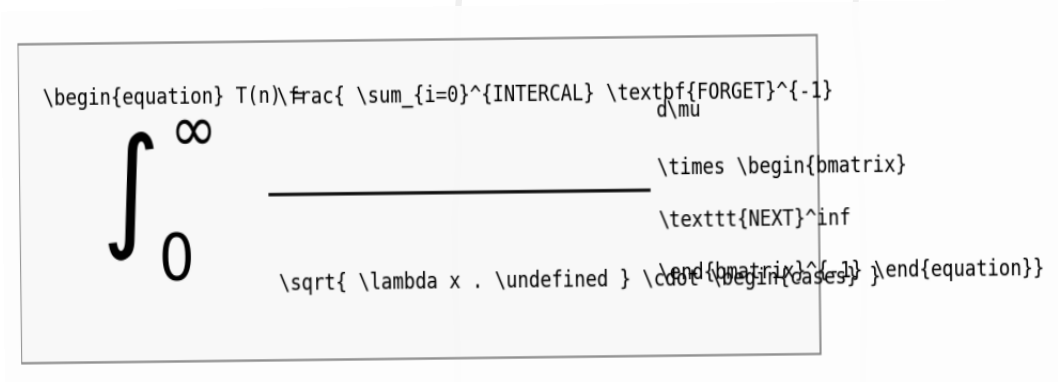


Figure 4: Runtime complexity of the POPCOUNT implementation. The authors note that this figure rendered correctly on their machine.

Bitwise AND is performed using the syslib's AND64 routine. Each step requires two AND64 calls (masking) and one ADD64 call (accumulation), for a total of 12 AND64 and 6 ADD64 invocations per POPCOUNT.

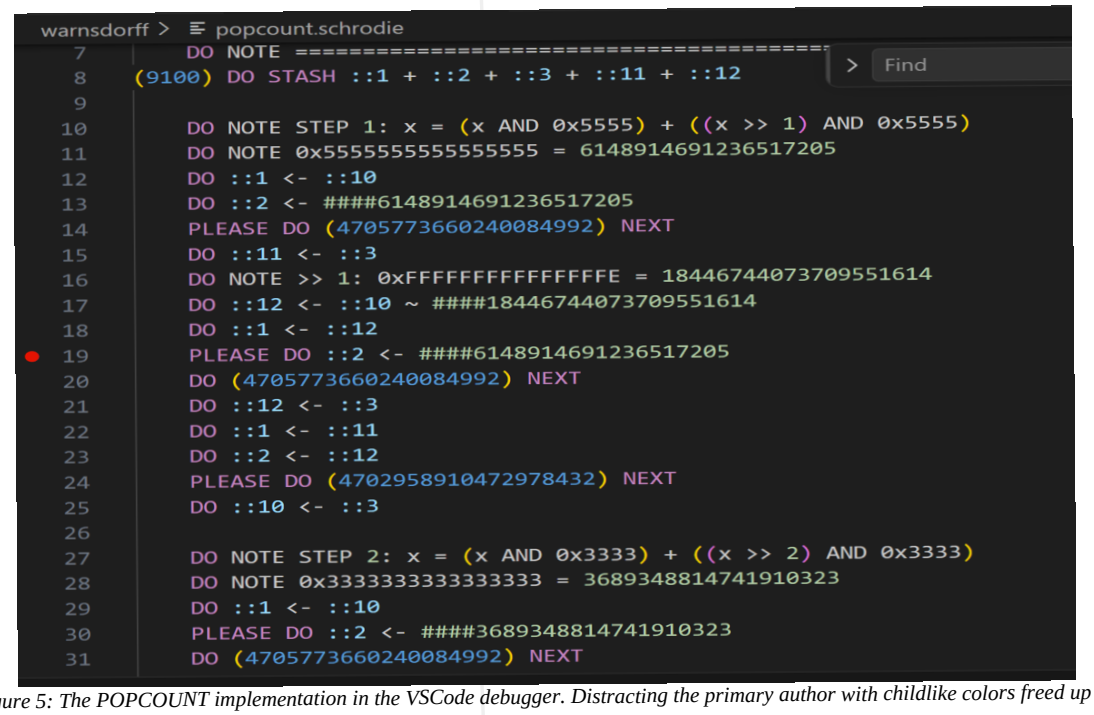


Figure 5: The POPCOUNT implementation in the VSCode debugger. Distracting the primary author with childlike colors freed up the secondary author to do the actual work required.

4.3 BIT-TO-INDEX

Converting a single set bit (a power of 2) to its bit index (0-63) uses the identity:

```
index = popcount(x - 1)
```

For a power of 2, x - 1 produces a value with all bits below x set, and the popcount of that value equals the bit position. However, the standard library's 64-bit subtraction routine (MINUS64) also contains a defect — the overflow handler triggers incorrectly during the complement-and-add operation. We work around this by computing x - 1 as ADD64(x, NOT64(0)), exploiting the unsigned arithmetic identity -1 = NOT(0) (mod 2^64).

4.4 LOWBIT (Lowest Set Bit Extraction)

Isolating the lowest set bit of a value uses the two's complement identity:

```
lowbit(x) = x AND (-x) = x AND (NOT(x) + 1)
```

This is implemented as three subroutine calls: NOT64, ADD64, AND64.

5. Control Flow

INTERCAL's execution model presents perhaps the greatest challenge to implementing iterative algorithms. The language provides:

```
NEXT      -- pushes a return address and transfers control to a labeled statement
RESUME #N -- pops N entries from the return stack and transfers control
FORGET #N -- discards N entries from the return stack without transferring
ABSTAIN FROM (label) -- causes a labeled statement to be skipped when reached
REINSTATE (label)    -- reverses a prior ABSTAIN
```

There are no loops, no conditional branches, and no comparison operators.

5.1 Conditional Branching via Double-NEXT Trampoline

The standard INTERCAL idiom for conditional execution uses a "double-NEXT trampoline." Given a flag variable .1 with value 1 or 2:

```
DO (A) NEXT          <- push return address R1
...                   <- path taken when .1 = 2 (RESUME pops R1 and R2)

(A) DO (B) NEXT      <- push return address R2
...                   <- path taken when .1 = 1 (RESUME pops only R2)

(B) DO RESUME .1      <- pops 1 or 2 entries
```

When .1 = 1, RESUME #1 pops one entry and returns to the code inside (A). When .1 = 2, RESUME #2 pops two entries and returns to the code after the (A) NEXT call. The flag value is produced by the zero-test idiom:

```
.1 <- "?'.1~.1'$#1"~#3
```

This evaluates to 1 when .1 is zero and 2 when .1 is nonzero.

5.2 The Fall-Through Problem

A critical subtlety of the double-NEXT trampoline: after the branch resolves, execution continues sequentially and encounters the DO RESUME .1 statement at label (B). If this statement executes a second time, it pops stack entries belonging to an outer scope, corrupting the control flow. Our solution uses ABSTAIN/REINSTATE to neutralize the trampoline after use:

```
(A) DO (B) NEXT
    DO ABSTAIN FROM (B)    <- prevent re-execution
(B) DO RESUME .1           <- skipped on fall-through
    ...                    <- sequential code continues safely
```

At the top of each loop iteration, REINSTATE (B) restores the trampoline for the next branch.

5.3 Loops via FORGET

INTERCAL loops are implemented by recursively calling a label with NEXT, using FORGET #1 at the top to discard the return address and prevent stack overflow:

```
(LOOP) DO FORGET #1       <- discard return from previous iteration
       ...                <- loop body
       DO (LOOP) NEXT     <- recurse (pushes new return address)
```

However, the initial FORGET also discards the return address from the caller on the first iteration. This means the loop cannot return to its caller via RESUME. Our solution: structure the program as a single linear flow. When the loop's exit condition is met, execution falls through to the next section of code rather than returning to a caller. This architectural constraint shaped the entire program structure. The candidate collection loop, minimum-finding loop, and tie-collection loop are all inline — they cannot be extracted as subroutines because INTERCAL's stack discipline makes returning from a FORGET-based loop impossible without corrupting the call stack.

5.4 Comparison Without Comparison

INTERCAL provides no comparison operators. We compare two values A and B (both in the range 0-128) using a safe subtraction technique:

```
1. Compute A + 128 - B using wrapping addition (1000) and subtraction (1010).
2. If the result has bit 7 set (value >= 128), then A >= B.
3. The bit check uses ~ #128 (select bit 7), followed by the zero test.
```

The offset of 128 prevents underflow in the subtraction, which would otherwise trigger a fatal runtime error (INTERCAL's subtraction routine aborts on underflow rather than wrapping).

6. Program Structure

The complete program consists of eight source files:

File	Purpose	Lines
knight_attacks.i	Attack mask lookup table	68
clear_mask.i	Bit-clearing mask lookup table	68
center_dist.i	Center distance lookup table	68
warnsdorff.i	Main program	170
my_add64.i	64-bit addition	60
popcount.i	64-bit population count	95
bit_to_index.i	Bit position extraction	20
lowbit.i	Lowest set bit isolation	20

Compilation order matters: data tables must appear first (their initialization code runs at program start), followed by the main program, followed by subroutine libraries (reached only via NEXT calls, never by sequential execution).

```
intercal.exe knight_attacks.i clear_mask.i center_dist.i \
  warnsdorff.i lowbit.i popcount.i \
  bit_to_index.i my_add64.i \
  -b -r:syslib64.dll -noplease
```

The main program flows linearly through five phases on each iteration:

```
1. Output and termination check -- print current square, increment counter, exit if 64.
2. Board update -- clear current square from the available mask.
3. Candidate collection -- extract each set bit from the candidate mask,
   compute its composite Warnsdorff/distance score, store in arrays.
4. Minimum search -- single pass over the score array, tracking minimum
   value and its index.
5. Position update -- look up the winning candidate's bit, convert to
   array index, loop.
```

7. Results

The program produces a complete knight's tour starting from e4 (square index 29, 1-based):

```
29 39 56 62 52 58 41 51 57 42 59 49 34 17  2 12
 6 16 31 48 63 46 61 55 40 23  8 14 24  7 13  3
 9 19 25 10  4 21 15 32 38 53 36 26 11  1 18 33
27 44 50 35 45 60 54 64 47 30 20  5 22 37 43 28
```

All 64 squares (1 through 64) appear exactly once. The program terminates cleanly after the 64th square is output.

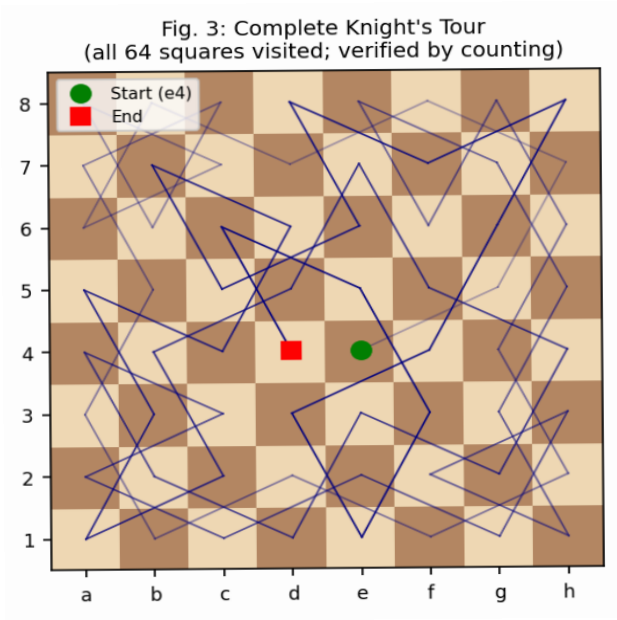


Figure 3: Complete knight's tour computed by the program, visualized on the board. The tour begins at e4 (green circle) and visits all 64 squares. Line opacity increases with move number, tracing the characteristic edge-hugging spiral consistent with the Arnd Roth amendment.

7.1 Performance Characteristics

The program's computational cost is dominated by POPCOUNT calls. Each candidate evaluation requires one POPCOUNT (6 ADD64 + 12 AND64 calls internally). With an average of 4-5 candidates per move and 64 moves, the program executes approximately 300 POPCOUNT calls, or roughly 1,800 ADD64 and 3,600 AND64 calls. Each ADD64 itself requires 8 calls to the 32-bit addition routine (1500) plus mingle and pack operations. Total subroutine call depth during scoring: main → POPCOUNT → AND64/ADD64 → (1500) → (1009), approximately 5 levels. INTERCAL's NEXT stack limit of 80 entries is never approached. The program executes in approximately 30 seconds on modern hardware. Performance optimization was not a design goal.

7.2 Defects Encountered and Workarounds

Defect	Impact	Workaround
Syslib ADD64 never writes output variable	All 64-bit addition fails silently	Reimplemented; local label shadows library
Syslib MINUS64 overflow on valid inputs	Cannot subtract any 64-bit values	Replaced x-1 with ADD64(x, NOT64(0))
RESUME trampoline fall-through	Infinite loops or stack corruption	ABSTAIN FROM handler after use; REINSTATE at loop top
FORGET destroys caller's return address	Subroutines with loops cannot return	Inlined all loops; linear program structure
Compiler parses labels in NOTE comments	Duplicate label errors	Avoid parenthesized numbers in comments
UINT64_MAX as ADD64 input causes runtime E666	Sentinel value cannot be used as arithmetic operand	Avoided as user-facing value
Compilation file order determines execution order	Table initialization must run before main program	Tables listed first; subroutines listed last

8. Conclusions

The implementation of Warnsdorff's algorithm in INTERCAL required solving three categories of problems: arithmetic (building 64-bit operations from 16-bit primitives), algorithmic (bitboard representation, composite scoring, lookup tables), and linguistic (INTERCAL's control flow model, stack discipline, and standard library defects).

The most significant insight was that INTERCAL's select operator (~), normally used for bit extraction, can implement logical right-shift when given a contiguous high-bit mask. This enabled the standard parallel POPCOUNT algorithm without any native shift support.

The ABSTAIN/REINSTATE mechanism, while designed for INTERCAL's characteristic absurdity, proved to be a practical solution to the trampoline fall-through problem — effectively implementing a primitive form of code patching at runtime.

The complete program, including all arithmetic subroutines and lookup tables, totals approximately 570 lines of INTERCAL source across eight files. It compiles to a .NET assembly and executes in approximately 30 seconds on modern hardware. The program is believed to be the most complex algorithmic implementation in INTERCAL to date.[2]

[2] *One of the authors suggests the commit history of the repository suggests future work in the ethics of attribution and authorship credit will be sorely needed.*

Acknowledgments

The center-distance tiebreaking amendment is due to Arnd Roth of the Max Planck Institute for Medical Research, Heidelberg, as documented by Gunno Tornberg. The original INTERCAL standard library routines for 16-bit arithmetic are derived from the C-INTERCAL distribution by Eric S. Raymond and others. The INTERCAL politeness checker, which rejects programs deemed insufficiently courteous, remains unchanged and was not modified during this work despite considerable provocation.

The arithmetic library, POPCOUNT algorithm, and main program were developed collaboratively between Jason Whittington and Claude (Anthropic). The authors thank quantum tiebreaking, which did not work, for clarifying what the correct solution was by being the wrong solution first.

References

Knuth, D. E. *A Song of Ice and Fire*. Addison-Wesley. *The Winds of Winter*: forthcoming.

Raymond, E. S. *The C-INTERCAL Reference Manual*. 1990-present. catb.org/esr/intercal.

Roth, A. "The Problem of the Knight." Max Planck Institute for Medical Research. web1.mpimf-heidelberg.mpg.de.

Tornberg, G. "Efficiency of Warnsdorff's Rule." Archived at web.archive.org.

Calvelli, C. (2001). CLC-INTERCAL. Available at http://www.zompower.tk/.

Squirrel, P. and Raymond, E. S. "INTERCAL Resources Page." catb.org/esr/intercal.

Stross, C. (2015). Published for the first time: the Princeton INTERCAL compiler's source code. esoteric.codes.

Whittington, J. (2019). CRINGE: Common Runtime INTERCAL Next-Generation Engine. github.com/jawhitti/INTERCAL.

Warnsdorff, H. C. *Des Rosselsprunges einfachste und allgemeinste Losung*. Schmalkalden, 1823.

Woods, D. R. and Lyon, J. M. *The INTERCAL Programming Language Reference Manual*. Princeton University, 1972.

Whittington, J. (2019). A Preliminary Investigation into Whether INTERCAL Could Be Made Worse. Unpublished manuscript, never submitted.